
Java Swing

Do zero a uma aplicação desktop funcional



Prof. Dr. João Choma

UEM

github.com/JoaoChoma

Fundamentos



Ao final da aula, você deverá ser capaz de:

- Explicar a estrutura de uma aplicação Swing.
- Criar janelas e componentes visuais.
- Organizar componentes com layouts.
- Reagir a eventos e validar entradas.
- Separar interface e regra de negócio.
- Construir uma calculadora desktop funcional.



```
Digite um numero: 10  
Digite outro numero: 20  
Resultado: 30
```

Console	Interface gráfica
Interação linear	Interação por componentes
Comandos digitados	Botões, campos e menus
Pouco feedback visual	Estado visível na janela
Ferramentas técnicas	Mais acessível ao usuário final



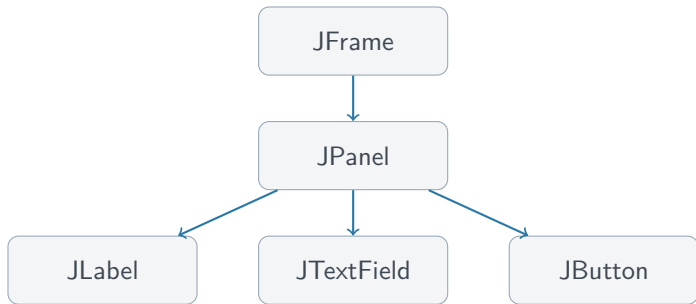
Swing é uma biblioteca da plataforma Java para construir interfaces gráficas desktop.

- Janelas e caixas de diálogo.
- Botões e campos de texto.
- Menus, listas e tabelas.
- Formulários e aplicações completas.

Os componentes ficam principalmente no pacote `javax.swing` e funcionam em Windows, Linux e macOS.

Aprendizado

Swing é um bom ambiente para estudar programação orientada a eventos e organização de aplicações desktop.



`JFrame` janela principal.

`JPanel` contêiner para organizar componentes.

`JLabel`/`JTextField`/`JButton` saída, entrada e ação.



Swing processa eventos e atualiza componentes em uma thread própria: a EDT.

```
public static void main(String[] args) {  
    SwingUtilities.invokeLater(() -> {  
        criarEExibirInterface();  
    });  
}
```

- Crie e atualize componentes Swing na EDT.
- Não execute tarefas demoradas nela: a janela congelará.
- Para tarefas longas, considere `SwingWorker`.



```
import javax.swing.*;

public class Main {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            JFrame janela = new JFrame("Minha Janela");
            janela.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            janela.add(new JLabel("Ola, Swing!"));
            janela.pack();
            janela.setLocationRelativeTo(null);
            janela.setVisible(true);
        });
    }
}
```

`pack()` calcula o tamanho necessário a partir dos componentes e do layout.



- 1 Criar o JFrame.
- 2 Configurar o fechamento.
- 3 Criar painéis e componentes.
- 4 Adicionar os componentes.
- 5 Executar `pack()`.
- 6 Centralizar com `setLocationRelativeTo(null)`.
- 7 Exibir com `setVisible(true)`.

Ordem importante

Torne a janela visível somente depois de concluir a montagem inicial.

Componentes e layouts



```
JLabel rotulo = new JLabel("Nome:");  
JTextField campo = new JTextField(20);  
JLabel resultado = new JLabel("Aguardando entrada");  
  
String nome = campo.getText().trim();  
resultado.setText("Ola, " + nome);
```

`getText()` lê o conteúdo do campo.

`setText()` atualiza o texto exibido.

`trim()` remove espaços nas extremidades.



```
JButton botao = new JButton("Calcular");  
  
botao.addActionListener(evento -> {  
    System.out.println("Botao clicado");  
});
```





Um painel agrupa componentes; seu Layout Manager decide tamanho e posição.

```
JPanel painel = new JPanel();  
painel.add(new JLabel("Nome:"));  
painel.add(new JTextField(20));  
janela.add(painel);
```

Evite coordenadas fixas

`setLayout(null)` quebra facilmente com fontes, sistemas operacionais e tamanhos de janela diferentes.



Layout	Uso típico
FlowLayout	Componentes em sequência
BorderLayout	Norte, sul, leste, oeste e centro
GridLayout	Grade com células do mesmo tamanho
BoxLayout	Organização vertical ou horizontal

`new GridLayout(3, 2, 8, 8)` cria três linhas, duas colunas e espaçamento de oito pixels.

Calculadora funcional



- 1 Digitar dois números.
- 2 Clicar em Calcular ou pressionar Enter.
- 3 Validar os valores.
- 4 Exibir a soma ou uma mensagem de erro.

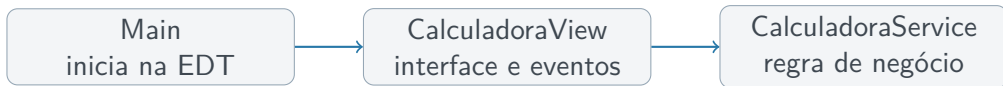
Interface planejada

Número 1: []

Número 2: []

[Calcular]

Resultado: --



Evite colocar interface, cálculo e validação em um único método `main`.



```
public class CalculadoraService {  
    public int somar(int a, int b) {  
        return a + b;  
    }  
}
```

- Reutilização da regra.
- Teste sem abrir uma janela.
- Interface gráfica mais simples.
- Apresentação pode mudar sem reescrever o cálculo.



```
public class CalculadoraView extends JFrame {
    private final JTextField numero1 = new JTextField(10);
    private final JTextField numero2 = new JTextField(10);
    private final JButton calcular = new JButton("Calcular");
    private final JLabel resultado = new JLabel("Resultado: --");
    private final CalculadoraService service;

    public CalculadoraView(CalculadoraService service) {
        super("Calculadora de Soma");
        this.service = service;
        configurarJanela();
        montarInterface();
        registrarEventos();
        pack();
        setLocationRelativeTo(null);
    }
}
```



```
private void montarInterface() {  
    JPanel painel = new JPanel(new GridLayout(3, 2, 8, 8));  
  
    painel.add(new JLabel("Numero 1:"));  
    painel.add(numero1);  
    painel.add(new JLabel("Numero 2:"));  
    painel.add(numero2);  
    painel.add(calcular);  
    painel.add(resultado);  
  
    painel.setBorder(  
        BorderFactory.createEmptyBorder(12, 12, 12, 12));  
    add(painel);  
}
```

Espaçamento e borda interna tornam o formulário mais legível.



```
private void configurarJanela() {  
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
    setResizable(false);  
}
```

Ao final do construtor, depois de montar a interface:

```
pack();  
setLocationRelativeTo(null);
```

Por que depois?

`pack()` precisa conhecer os componentes para calcular o tamanho da janela.



```
private void registrarEventos() {  
    calcular.addActionListener(e -> calcular());  
    numero2.addActionListener(e -> calcular());  
}  
  
private void calcular() {  
    int n1 = Integer.parseInt(numero1.getText().trim());  
    int n2 = Integer.parseInt(numero2.getText().trim());  
    int soma = service.somar(n1, n2);  
    resultado.setText("Resultado: " + soma);  
}
```

Um método nomeado evita duplicar a ação entre o botão e a tecla Enter.



```
private void calcular() {  
    try {  
        int n1 = Integer.parseInt(numero1.getText().trim());  
        int n2 = Integer.parseInt(numero2.getText().trim());  
        resultado.setText("Resultado: " + service.somar(n1, n2));  
    } catch (NumberFormatException e) {  
        JOptionPane.showMessageDialog(  
            this,  
            "Digite dois numeros inteiros validos.",  
            "Entrada invalida",  
            JOptionPane.ERROR_MESSAGE);  
        numero1.requestFocusInWindow();  
    }  
}
```

Capture a exceção específica, não `Exception` genericamente.



```
public class Main {  
    public static void main(String[] args) {  
        SwingUtilities.invokeLater(() -> {  
            CalculadoraService service = new CalculadoraService();  
            CalculadoraView view = new CalculadoraView(service);  
            view.setVisible(true);  
        });  
    }  
}
```

Main → View → evento → validação → Service → View

Qualidade e evolução



- Definir foco inicial no primeiro campo.
- Permitir cálculo com Enter.
- Exibir mensagens específicas e curtas.
- Usar bordas e espaçamentos consistentes.
- Não bloquear a EDT com tarefas demoradas.

Definindo o botão padrão da janela:

```
getRootPane().setDefaultButton(calcular);
```



```
class CalculadoraServiceTest {  
    @Test  
    void deveSomarDoisInteiros() {  
        CalculadoraService service = new CalculadoraService();  
  
        int resultado = service.somar(10, 20);  
  
        assertEquals(30, resultado);  
    }  
}
```

A separação entre View e Service permite testar a regra rapidamente.



Construa uma aplicação com:

- campos Nome e Categoria;
- botões Adicionar, Remover e Listar;
- armazenamento em `ArrayList<Jogo>`;
- mensagens de validação;
- lista visual com `JList` ou `JTable`.

`Main` → `JogoView` → `JogoService` → `ArrayList<Jogo>`

Próxima evolução

Persistir os jogos em arquivo ou banco de dados.



- JFrame é a janela; JPanel e layouts organizam.
- JLabel, JTextField e JButton constroem o formulário.
- ActionListener reage às ações; JOptionPane informa.
- A interface deve ser criada na EDT.
- A View apresenta; o Service executa as regras.
- Validação e feedback fazem parte da experiência.

Próximo passo

Construir aplicações CRUD completas utilizando Swing.